

Copart Vehicle Search

Design overview and submission notes

Sai Kiran Dasari · October 3, 2026

Live demo: copart-vehicle-search.up.railway.app

Repository: github.com/SaiKiran6305/copart-vehicle-search

AI usage record: AI_USAGE.md in the repository

This prototype is a responsive vehicle search experience backed by a paginated Spring Boot API. Keyword search and filters are deterministic. An **optional Ask AI feature turns plain-English requests**, including misspelled ones, into the same supported filters. All listing data is synthetic: the application demonstrates the search workflow, not live Copart inventory.

1. Problem and scope

The assignment was to build a vehicle search experience with a React frontend and a backend API. Reviewers can search sample listings, filter, sort, page through results, and try natural-language search. The application runs as one service on the Railway.

Assumptions

- Reviewers use a public sample catalog; search and filters cover the fields in the seeded dataset.
- Keyword search must keep working when AI is unavailable or not configured.
- Saved vehicles only need to persist in the visitor's browser (no accounts).

Included

- React search interface with filters, sorting, pagination, saved vehicles and shareable URLs.
- Spring Boot REST API with validation and clear error responses.
- H2 database with 1,000 synthetic vehicle records from a fixed-seed generator.
- Optional AI query interpretation with rate limits.

Out of scope

Live Copart inventory, bidding, checkout, user accounts, and durable production data storage. Vehicle photos, values and sale dates are illustrative or synthetic.

2. What the application does

- **Keyword search:** Each word is matched against the lot number, make, model and location, and a four-digit number also matches the year, so "toyota dallas" and "2018 camry" work. Filler words such as "in" and "near" are ignored.
- **Filters :** Make and model (linked), primary damage, condition, estimated value range and year range. Dropdowns apply immediately.
- **Sorting and paging:** Ten sort options; 12, 24, 48 or 96 results per page.
- **Ask AI:** Describe what you want in plain English. The AI fills in the supported filters, corrects obvious misspellings, and asks a short question when unsure. A "Did you mean ...?" The question can be answered with one click: Yes, or one button per choice.
- **Popular searches.** One-click chips for Under \$5,000, Tesla, 2025 or newer, and Texas. Clicking an active chip turns it off.

- **Shareable searches.** The applied search is kept in the URL, so links can be shared, a refresh keeps the results, and Back and Forward step through searches.
- **Saved vehicles.** Hearts are stored by lot number in the browser and stay in sync across tabs.
- **Vehicle cards.** Photo, year, lot number, primary damage, condition, sale date, a "Sale in N days" tag for sales within two weeks, odometer and estimated value.
- **Helpful states.** Skeleton cards while loading, the server's explanation when a request is invalid, and suggestions of which filter to remove when few vehicles match.
- **Responsive layout.** Four, three, two or one card columns by screen width; on phones the filters collapse behind a button, and a compact search bar follows the page as you scroll.

Screenshots of these features are in the appendix.

3. Architecture and design choices



Figure 1. Runtime architecture. One Spring Boot service serves the React app and the API; the AI provider is optional.

The search path is **VehicleController** → **VehicleService** → **VehicleRepository**, with **VehicleSearchSpecification** composing the optional filters. The AI path is **AiSearchController** → **AiSearchService** → **OpenAI Responses API**; the interpreted filters return to the browser, which runs them through the normal search endpoint.

Choice	Why it fits this prototype	Trade-off
One deployment	A multi-stage Dockerfile builds React, packages it into Spring Boot, and runs one Java service. UI and API share one origin, so no second service or CORS setup is needed.	The frontend and API are deployed and scaled together.
H2 and fixed-seed data	No database service to provision. A fixed-seed generator makes the 1,000 records reproducible, and the app reloads them at startup.	Not a durable store: the hosted data is recreated on each deployment.
JPA Specifications and paging	Optional filters, sorting and paging run in the database, and only the requested page is returned.	Keyword matching is word-by-word substring matching, not fuzzy or relevance-ranked.

AI only interprets	The model turns text into the existing filter schema; the regular search API still finds the vehicles, and its output is validated.	Needs an API key, adds 2–4 seconds, and can ask for clarification. Keyword search does not depend on it.
Search state in the URL	Searches can be shared and refreshed; Back and Forward work.	Only applied searches are stored, not unsent edits.
Pre-compressed, cached assets	The build writes Brotli and gzip copies; hashed files are cached for a year and index.html is re-checked on every visit.	One quick 304 check for index.html on repeat visits.

4. API

Endpoint	Purpose	Behavior
GET /api/health	Health check	Returns {"status":"UP"}.
GET /api/vehicles	Search lots	Keyword, filters, sorting and paging; returns one page plus totals (number, size, totalElements, totalPages, first, last).
POST /api/ai-search	Interpret text	Accepts query (1–300 characters) and an optional clarification. Returns READY with supported filters, or CLARIFICATION with a short question.

Search parameters: q, make, model, primaryDamage, condition, minYear and maxYear (inclusive), minPrice (inclusive) and maxPrice (exclusive), maxPriceInclusive (used by AI for "under \$X"), page (from 0), size (1–100, default 12), sortBy (year, make, model, saleDate, estimatedValue, odometer) and direction (asc or desc). Ties are ordered by id so paging is stable.

Errors: invalid requests return HTTP 400 as an RFC 9457 problem detail whose detail says what to fix, for example "size must be between 1 and 100." The UI shows that message. Ask AI returns 429 with Retry-After when rate is limited, 503 when no API key is configured, and 502 when the AI provider fails or times out.

5. Data

- 1,000 synthetic lots: 10 makes with 3 models each, 100 lots per make, LOT-1001 to LOT-2000, across 15 US cities.
- Model years 2016–2025; estimated values from \$2,950 to \$56,400, derived from age, mileage, primary damage and condition; sale dates on weekdays from October 7 to December 15, 2026.
- Generated by scripts/generate-vehicles.mjs with a fixed seed, so re-running it produces the same file. At startup the loader replaces stored rows if they differ from the seed file.
- At this size almost every two-filter search returns vehicles: model + damage is empty 6% of the time (34% with the earlier 300 records), and make + city never is.
- 25 illustrative photos in six body styles (sedan, hatchback, SUV, electric crossover, pickup, coupe), including color variants and mirrored copies. Each lot always shows the same photo, chosen from its lot number. Photos are not of the actual lot.

6. AI search design

1. The visitor types a request and selects Ask AI. The backend checks the rate limits: 10 requests per minute per client IP and 200 per hour overall.
2. The backend sends the text, with instructions listing the supported makes, models, damage types, conditions and cities, to OpenAI's gpt-5.6-luna model through the Responses API. It requires a strict JSON schema, caps the answer at 300 output tokens, sets store: false, and allows 15 seconds.
3. The model answers READY with filters, or CLARIFICATION with one short question. The instructions tell it to correct obvious misspellings ("Dalls" → Dallas, "corolle" → Corolla) and to ask only when a word could mean two values or matches nothing.
4. The backend checks every value against the catalog: known make and model, a model that belongs to the make, known damage and condition, years from 1886 to 2100, and a price limit up to \$1,000,000. Anything unexpected becomes a clarification question or an error, never an unchecked filter.
5. The browser fills in the filters, shows "AI interpreted" chips, and runs the regular search. Clarification questions show one-click answers when they are yes/no or offer two or three choices.

The API key exists only as a server environment variable and never reaches the browser. User text is treated as data, not instructions.

7. Performance

The first load was a chain: HTML, then an uncompressed 252 KB script, then the API request, then photos. Changes: Brotli/gzip copies written at build time (script 252 KB → 68 KB), long-lived caching for hashed files, an early default search started from index.html while the script downloads, and the first row of photos loaded immediately with high priority.

First visit (local Chrome, simulated network)	Before	After
Desktop, broadband	0.56 s	0.38 s
Desktop, fast 4G	0.98 s	0.64 s
Phone, fast 4G	0.89 s	0.61 s
Phone, slow 4G	3.1 s	1.9 s

Time until the first row of photos is visible. These are local test results, not production guarantees. On a slow phone network, image loading accounts for most of the remaining time.

8. Reliability and security

- OpenAI key stored only as a Railway variable; AI search rate-limited per client and overall.
- H2 web console disabled by default, so it is not exposed on the public site.
- Consistent RFC 9457 error responses for invalid input, including values of the wrong type.
- Idle connections to OpenAI are closed after 30 seconds, after a dropped connection made Ask AI time out (see challenges).
- No secrets in the repository.

9. Testing and verification

Frontend: 65 tests (Vitest and React Testing Library) covering keyword search and Enter, dropdown filters, clearing, sorting and paging, URL state and Back/Forward, saved vehicles, error messages, empty and sparse results, Ask AI including clarifications and one-click answers, popular-search chips, the sale countdown, photo selection, and the early first search.

Backend: 41 tests (JUnit 5, Spring Boot Test, MockMvc) covering multi-word search, filters and sorting, request validation and problem-detail errors, rate limiting, AI output validation, the seed loader, static-file caching and compression, and that the AI instructions list every make, model and city in the data.

CI: GitHub Actions runs the frontend tests, a production build and the backend tests on every pull request and push to main. Layouts were also checked in Chrome at phone, tablet and desktop widths.

10. Challenges and how they were resolved

One deployment for the frontend and backend

Railway initially could not detect how to build the separate frontend and backend, and the first successful deployment served the API without the UI. I added a multi-stage Dockerfile: Node builds React, Maven packages those assets with Spring Boot, and one Java service serves both. The UI and API now share one URL, with no separate frontend deployment or CORS setup.

Protecting the OpenAI key and limiting usage

When I added Ask AI, I needed to keep the API key out of the browser and prevent runaway usage. The key is stored as a Railway variable and used only by the backend. Per-client and application-wide limits return HTTP 429 with a Retry-After value. The limits are kept in memory, so they reset on restart and are not shared across instances.

Making keyword search work across fields

Searches like "toyota dallas" returned nothing because the whole phrase had to appear in one field. Each word can now match a different field, and a four-digit number also matches the year. After "model 3" matched unrelated lot numbers, short numbers stopped being matched against lot numbers. With the current data, "toyota dallas" returns 6 vehicles and "2018 camry" returns 5.

Ask AI stopped working after a quiet period

After about half an hour without AI searches, requests that normally took 2–4 seconds failed after 20 seconds. OpenAI reported no outage, while Railway's network logs showed the server re-sending its request on a connection that never answered. The network had silently dropped the idle connection, and the Java HTTP client kept reusing it. Idle connections are now closed after 30 seconds and the timeout is shorter, so the first search after a quiet period opens a fresh connection.

Reducing the time to show vehicle photos

The first results waited for the whole script to download before the API request even started, and the script was sent uncompressed. Pre-compressed files, cache headers, an early first search and prioritized first-row photos brought the time to the first photos from 0.56 to 0.38 seconds on desktop broadband, and from 3.1 to 1.9 seconds on a simulated slow phone network.

11. Known limitations

- All records are synthetic, and photos are illustrative, not of the actual lot.
- H2 is recreated on each deploy (no persistent volume); the seed data repopulates it at startup.
- Keyword search is not typo tolerant; Ask AI corrects obvious misspellings but depends on an external provider and may ask for clarification.
- Rate limits are in memory per running instance.
- Saved vehicles stay in one browser profile and do not follow a user to another device.

12. What I would do next

- Move to PostgreSQL with indexes the case-insensitive searches can use, plus full-text or fuzzy search for typo-tolerant keyword search.
- Share rate limits across instances (for example with Redis) and monitor AI latency and errors.
- Replace illustrative photos with real lot images and add a vehicle detail page.
- Serve static assets through a CDN and add user accounts so saved vehicles follow the user.

13. Use of AI assistants

ChatGPT (GPT-6 Sol) helped with hosting and architecture decisions, Claude (Sonnet 5.5) with seed data, boilerplate and test drafts, and Claude Code (Claude Opus 5.5) with reviews, fixes, features, tests and performance work. Every change went through a pull request and CI and was reviewed before merging. **AI_USAGE.md** in the repository records the models, prompts and how responses were used.

Appendix: Screenshots

Captures of the deployed application. The listings are synthetic demonstration records.

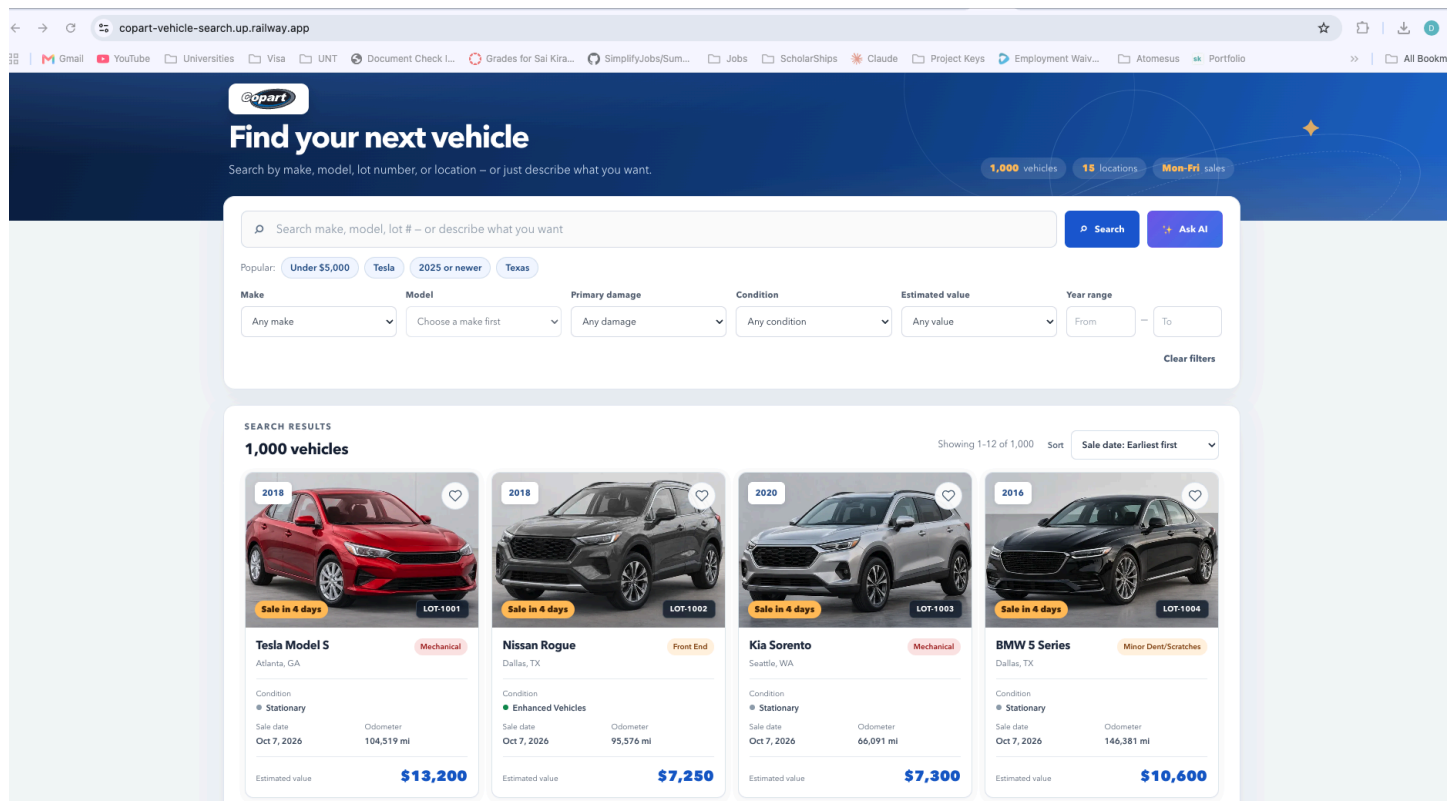


Figure 2. Default search: 1,000 synthetic vehicles, 12 per page, sorted by earliest sale date.

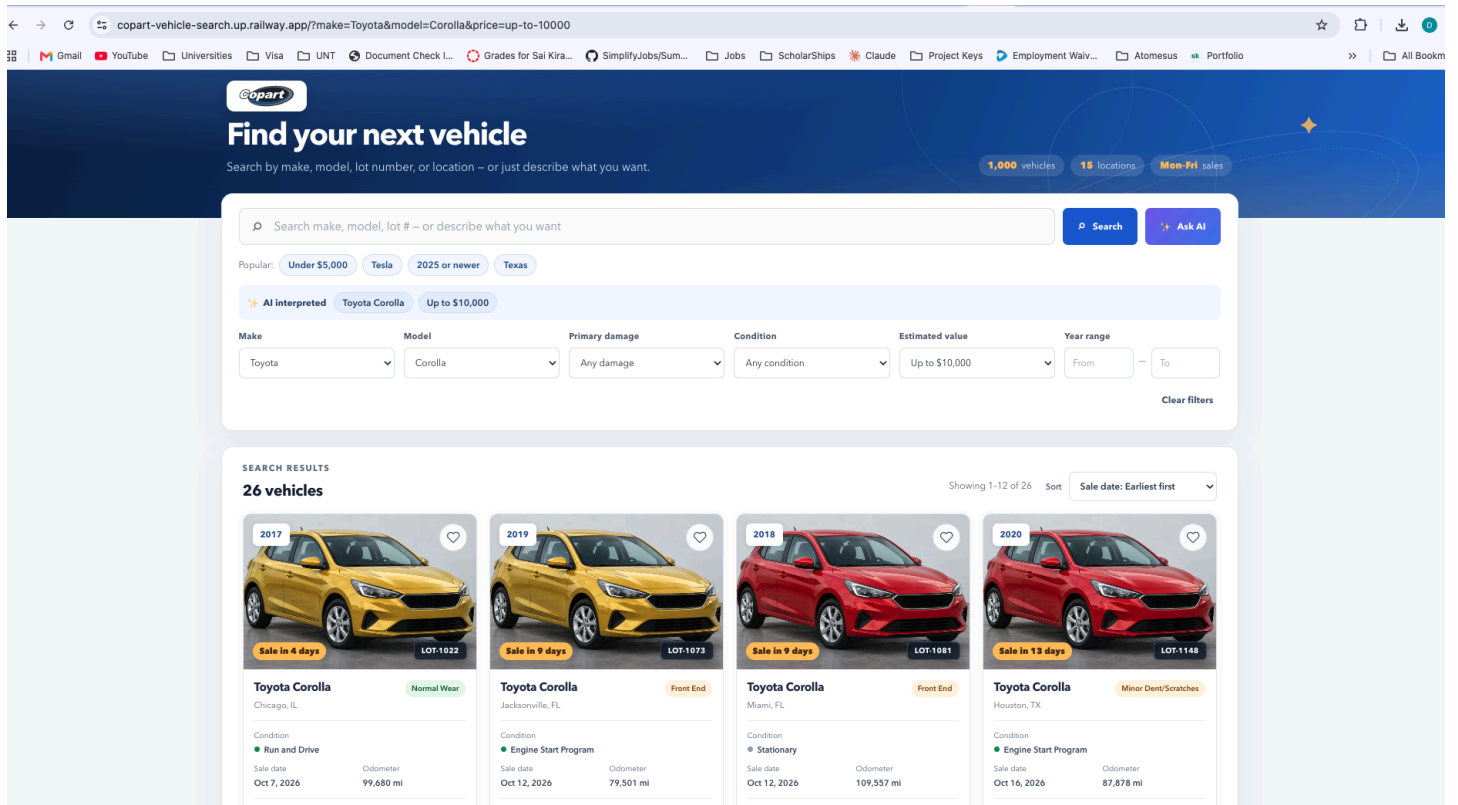


Figure 3. Ask AI corrects "corolle" to Corolla and applies the price limit; the regular search API returns the matches.

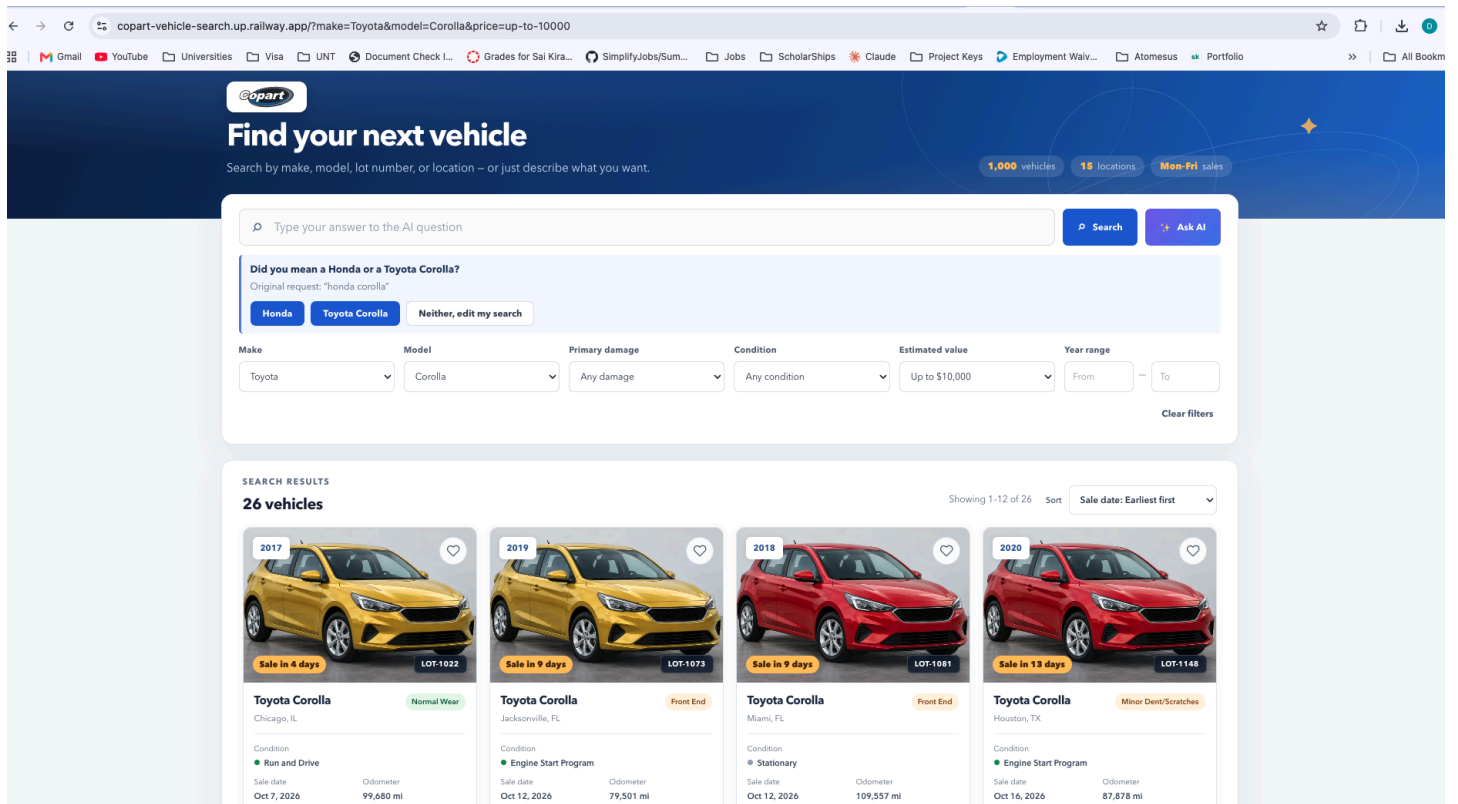


Figure 4. When the request is ambiguous, Ask AI asks one question with one-click answers.

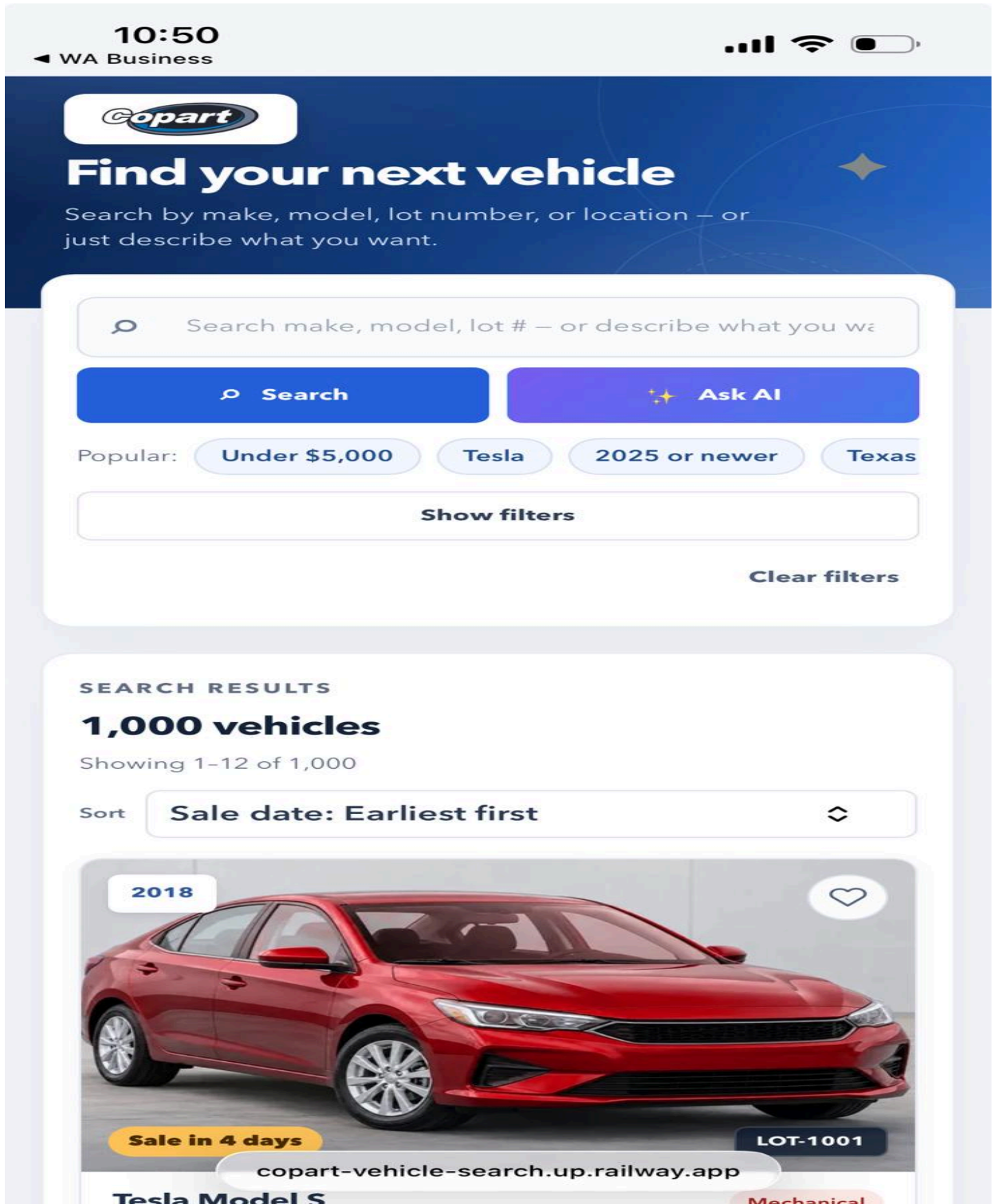


Figure 5. Mobile layout: filters collapse behind "Show filters", popular searches scroll sideways, and cards use one column.